

PRACTICA_02 motores (Reporte)

Nombre: Elliot Kenneth Villafaña Pasten

Usuario de GitHub: Piabonnie-Dev

Objetivo de la práctica

Implementar una estructura matemática Vector2 en C++ (con suma, resta, escala, magnitud y normalización), capturar entrada continua de teclado con `SDL_GetKeyboardState` bajo el sistema de callbacks de SDL3, modularizar el código en un archivo header (`Vector2.hpp`), organizar el estado del jugador mediante composición (`struct Character`), compilar el proyecto con CMake y corregir el error del movimiento diagonal.

EL ERROR DEL MOVIMIENTO DIAGONAL (CAUSA Y CORRECCION)

Cuando el jugador presiona una tecla de dirección, el vector de entrada tiene magnitud 1: p.e.j, al moverse hacia la derecha $v=(1,0)$, con $||v|| = \text{raíz}(1^2 + 0^2) = 1$. El desplazamiento resultante es exactamente $\text{velocidad} \times \Delta t$, es decir, 300 px/s.

El problema aparece cuando se presionan dos teclas perpendiculares al mismo tiempo, por ejemplo W y D para moverse en diagonal hacia arriba-derecha. El vector de entrada compuesto es $v = (1, -1)$, y su magnitud ya no es 1:

```
nv_diagn = raíz(12 + (-1)2) = raíz2 » 1.4142
```

Si este vector se usa directamente para mover al personaje, el desplazamiento en diagonal resulta un 41.4% más largo que el movimiento en línea recta: $1.414 \times 300 \gg 424.26$ px/s en vez de los 300 px/s configurados.

La solución es normalizar el vector de entrada antes de escalarlo por la velocidad: dividir cada componente entre la magnitud del vector para que esta quede en exactamente 1, conservando la dirección original:

```
vn_diag = (1/raiz2, -1/raiz2) » (0.7071, -0.7071) , nvn_diagn = 1
```

En el código, esto se implementó en `Vector2::normalized()` (con protección contra división entre cero

usando un umbral de 0.0001f) y se aplica en `SDL_Applterate` antes de calcular el desplazamiento: `if (input_dir.length_squared() > 0.0f) input_dir =`

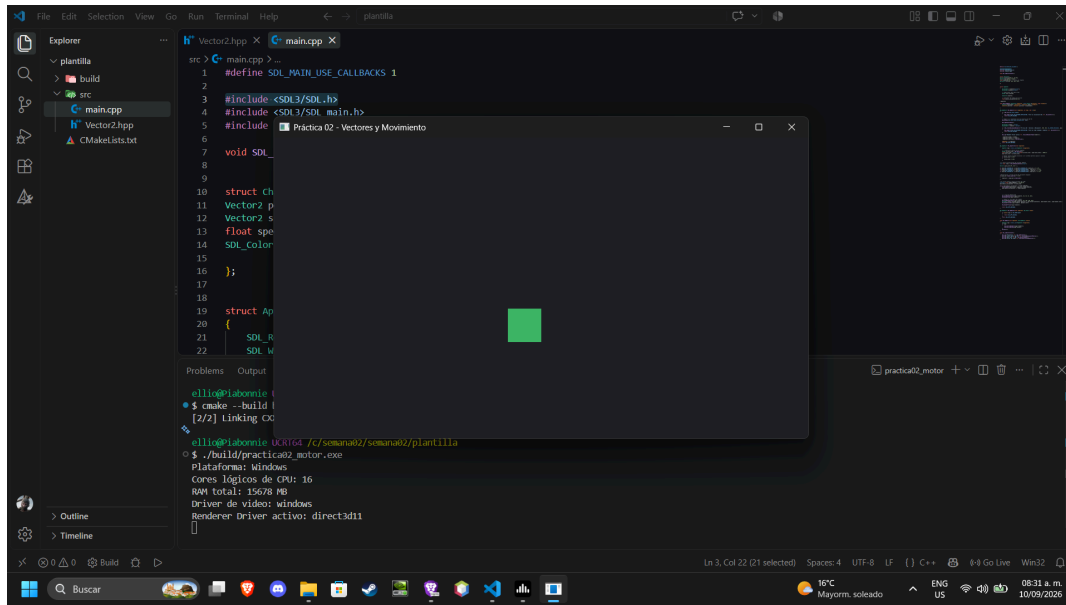
```
input_dir.normalized();.
```

Con esta corrección, el jugador se desplaza siempre a 300 px/s sin importar si se mueve en línea recta o en diagonal.

Arquitectura del código

- **Vector2.hpp**: estructura matemática independiente (operator+, operator-, operator*, length(), length_squared(), normalized()), protegida con #pragma once.
- **struct Character**: agrupa por composición la posición, tamaño, velocidad y color del jugador ("un personaje TIENE un Vector2", no ES uno).
- **Fixed Timestep**: la física se actualiza en pasos constantes de 1/60 s mediante un acumulador de tiempo, desacoplada del Dt variable del renderizado.

Captura de pantalla: ventana en ejecución



Commit/Repositorio en GitHub

<https://github.com/Piabonnie-Dev/practica02-motor.git>